

React → JS library

SI → ①

- Reusable components
- faster UI updates
- State management
- large community
- Industry
 - Netflix
 - Airbnb
 - Uber

Vanilla JS → logic UI

- ↳ Manually DOM Manipulation
- ↳ Manually update

```
<html>
  <body>
    <h1>
    </h1>
  </body>
</html>
```

Document
Object
Model

```
HTML
css
-----
JavaScript
```

② Code Becomes messy

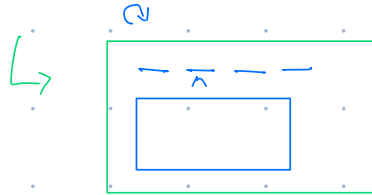
③ Re-usability

④ DOM - performance

↳ Virtual DOM → Structure webpage
↳ light weight copy

```
State Change
  ↓
Virtual DOM
  ↓
Old vs New
  ↓
Update
  ↓
Re-render
```

Single Page Application (SPA)



Component Based Architecture

→ Component - Independent → Reusable UI

APP

- Navbar
- Sidebar
- product card x100
- Button - []

- ✓ <Button />
- ✓ 2 Button />
- ✓ <Button />

Important React

①

JSX (Javascript XML)

↳ HTML like syntax

②

Component - Building Blocks

③

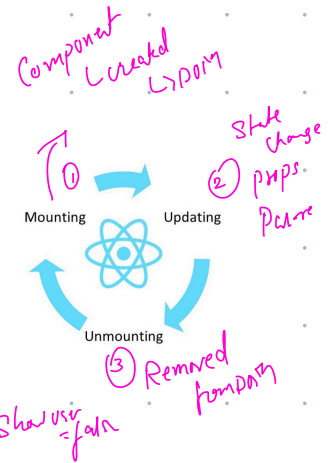
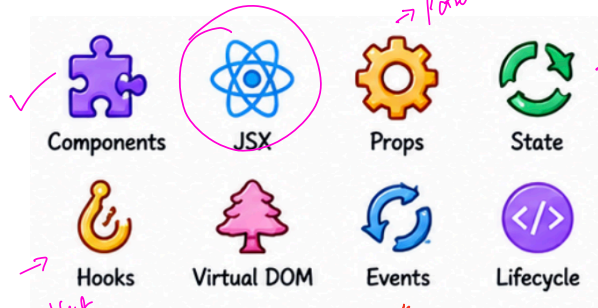
Props - properties - Allow data to pass
Parent - child component

④ State → Stores data - change Component
↳ useState - React

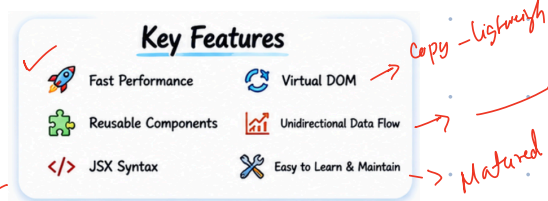
⑤ Events → interaction - Handled by Events
Functional Component - React hooks

⑥ Hooks →
⑦ React Router → Navigation

Class Component - state
① Functional Component - useState
(React Hooks)



function
Component didMount ✓
didUpdate ✓
willUnmount ✓
useEffect
AngularJS → framework
ReactJS → library
MEAN
MERN



HTML - JS

< welcome / >

The fundamental building blocks you need to know.

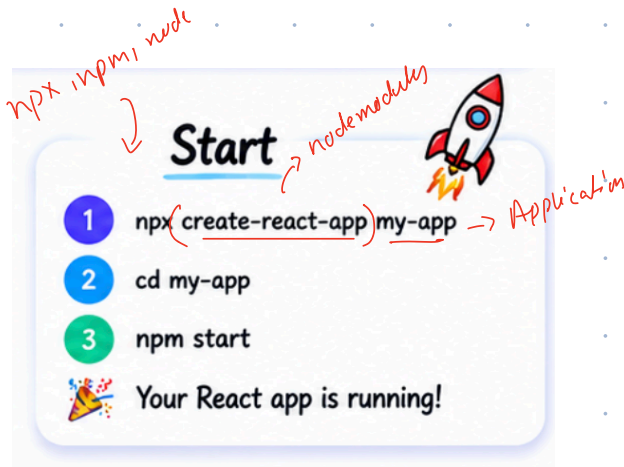
	1. Components Components are independent, reusable pieces of UI. They can be functional or class-based.	<pre>function Welcome() { return <h1>Hello, React!</h1>; } export default Welcome;</pre> → components
	2. JSX (JavaScript XML) JSX allows us to write HTML-like code inside JavaScript. It is transformed into React elements.	<pre>const element = <h1 className="title"> Hello, JSX! </h1>;</pre>
	3. Props (Properties) Props are inputs to components. They are passed from parent to child and are read-only.	<pre>function Greet({ name }) { return <p>Hello, {name}</p>; } <Greet name="React Developer" /></pre>
	4. State State is data that can change over time within a component. When state changes, the UI re-renders.	<pre>const [count, setCount] = useState(0); setCount(count + 1); // UI updates automatically</pre>
	5. Events React handles events similarly to HTML events, but with camelCase naming.	<pre>function Button() { return <button onClick={() => alert('Clicked!')}> Click Me </button>; }</pre> → click onclick onClick()

Getting Started

- ① Node.js
- ② npm
- ③ Vite

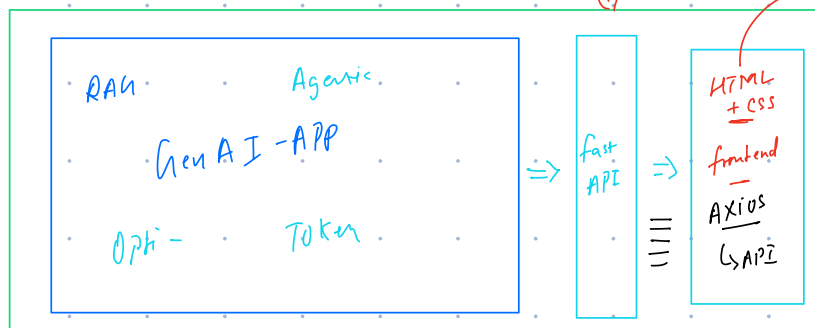
Hot Reloading

```
my-react-app/  
├── node_modules/ ✓npm  
├── public/  
├── src/  
│   ├── assets/  
│   ├── App.jsx  
│   └── main.jsx  
├── package.json  
├── vite.config.js  
└── index.html
```

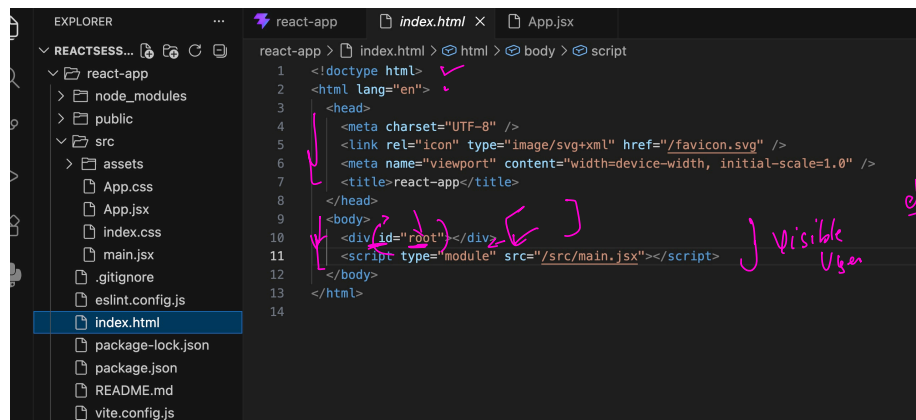
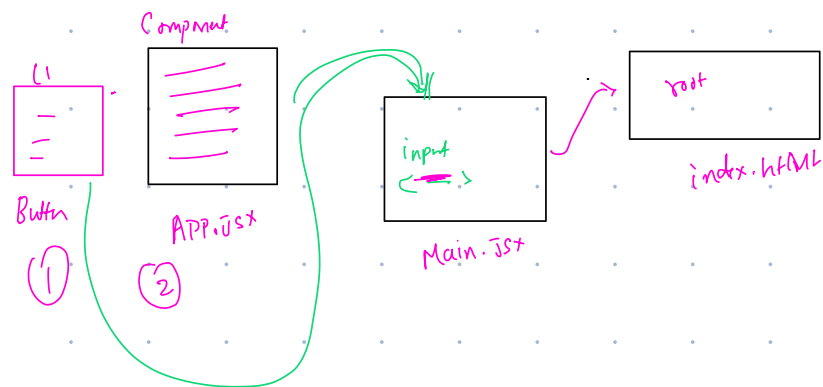
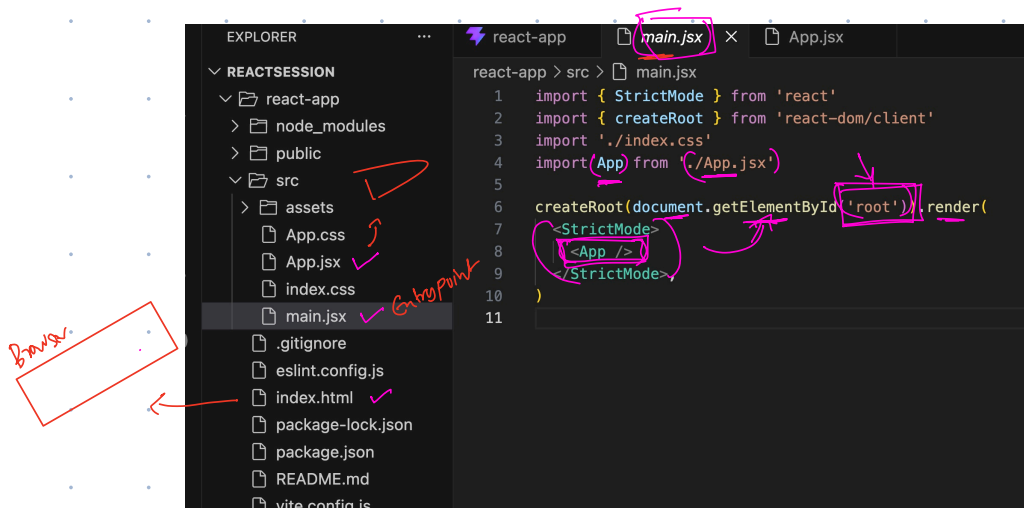


GenAI → Frontend - FDE → Frontend

API - Endpoints

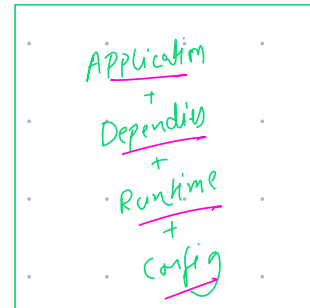


UI layer





Docker → Containerization Platform

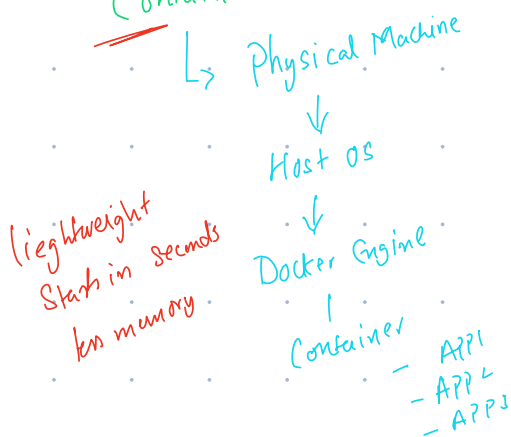


laptop — Runs Same
Flaptop — Container
↳ Portable unit

- ↓
- ① it works on my machine
 - ② Different OS Versions
 - ③ Python / Node
 - ④ Dependency conflicts
 - ⑤ Difficulty Deployments

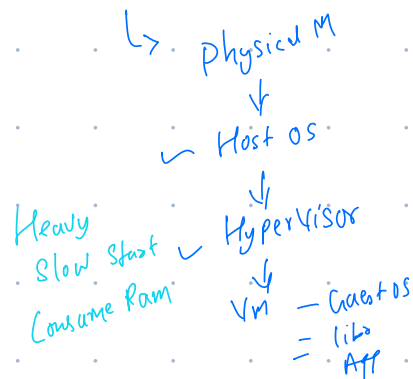
Packages

Container



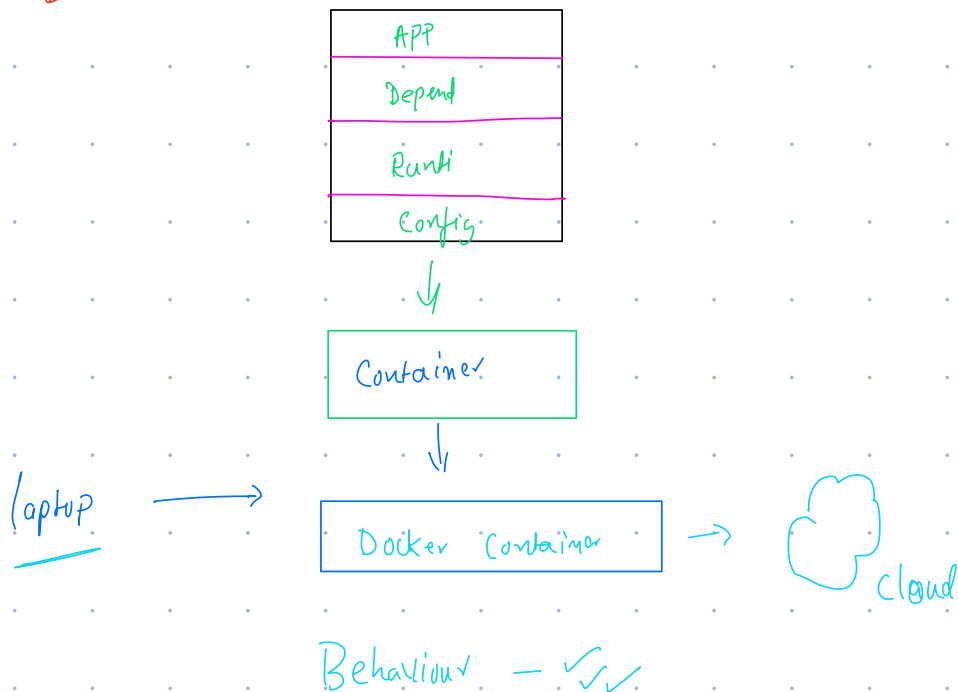
lightweight
Starts in seconds
low memory

Virtual Machine



Heavy
Slow Start
Consume Ram

Docker



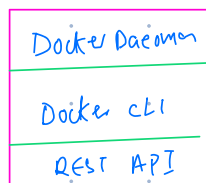
Docker

① Docker Engine

Core Service → Running Containers

Background Service
↳ create and manage containers

You
↓
CLI
↓
Daemon
↓
Container



→ Command line
↳ docker run
docker build
docker ps
docker images

② Docker Image

↳ Blueprint / Template

↳ Create Container

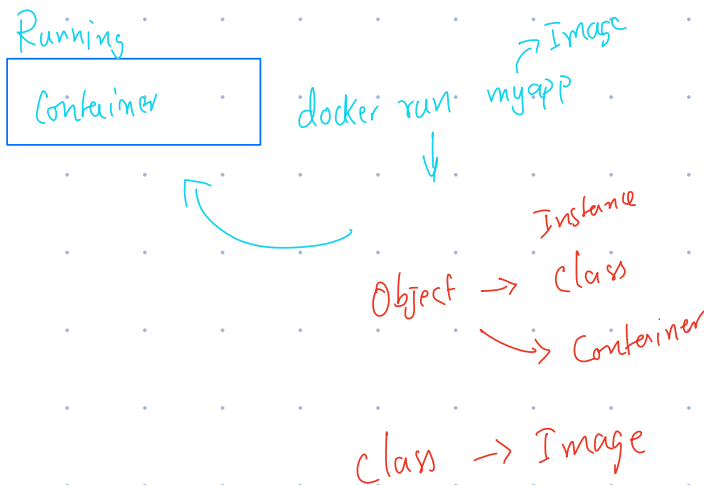
→

Python 3.12
+
API
+
code

Class → Image

③ Docker Container

Running Instance of Image



④ Docker Hub → Public Docker Image Repository
Docker Images

- Python
- node
- mongo
- nfn

- ⑤ Docker Desktop → GUI App
 Multiple Containers → Projects
- ⑥ Docker Compose → Compose.Yaml
 Run - M-C - One single

Docker Workflow

Docker hub
Image



Docker
Image
Template

(class)

Docker Run



Docker
Container
(Running)

Running

(objects)

Docker Commands

```

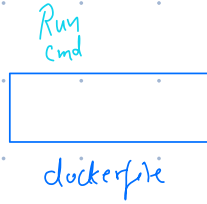
docker --version
images - list all images
ps -> Running
ps -a -> Containers list
Pull <image> -> docker hub - pull
build -t <name> -> Image dockerfile
run <image> -> Run a container
stop
start
  
```

rm
rmi <imageid>

Dockerfile

(create images)

Cloned



build -t



docker image
Template

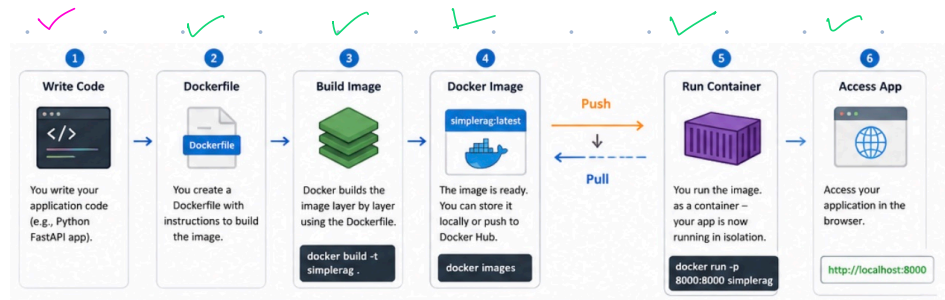
Run



Container

[8080
cmd.]

Simple RAG Demo →



Requirement

Docker Installed - ✓